

Diagnose and Improve an Agent's Behavior Using Logs and Edge Cases

Edge Case

In this hands-on activity, you'll take the role of an agent designer diagnosing a behavioral breakdown. You'll be given a scenario from an AI agent that technically runs—but does not behave as expected. Your task is to analyze what went wrong, identify the weak point, and propose a meaningful fix to improve its performance.

This lab focuses on developing your agent debugging mindset: learning to evaluate logic, memory use, and loop flow—not just final outputs.

Objective

This activity builds your skill in diagnosing agent behavior—not just outputs, but the logic, memory, and flow that drive them. By analyzing a broken interaction, you'll sharpen your ability to spot weak links and design smarter, more resilient agents. Is your agent functional—or truly reliable? This is your step toward mastering intelligent system troubleshooting.

Scenario

The Travel Planner Agent That Loops and Forgets

Agent Description:

You've designed a deliberative travel planner agent using LangChain. Its task is to:

- Ask the user for their destination

- Suggest 3 hotel options

- Ask for confirmation

- Proceed to book the selected hotel

- Send a confirmation message with booking details

It uses a basic memory module to track conversation history and user responses.

Observed Behavior (from logs; e.g., chain-of-thought traces, action outputs)

- The agent correctly asks: “Where would you like to travel?”
- User responds: “I’d like to go to Barcelona.”
- Agent suggests 3 hotel options (✓)
- User: “Let’s go with the second one.”
- Agent: “Where would you like to travel?”
- User: “I already told you—Barcelona.”

Agent repeats the 3 hotel options.

User: "I still want the second one."

Agent: "Where would you like to travel?"

Agent exits: "Sorry, I couldn't complete your booking."

Instructions for learners

Your Task

Based on this scenario:

Diagnose the issue. What likely caused the loop and memory failure?

Identify the weak link. Was it poor memory configuration? Faulty logic sequencing? Weak intent matching?

Propose an improvement. What change would prevent this breakdown?
Could it be a better fallback handler, improved slot filling, memory persistence fix, etc.?

Submission

What to Submit

- 2–3 sentence diagnosis of what went wrong
- 1–2 sentence design improvement recommendation
- (Optional) A sketch of a logic flow or a pseudocode fix

Conclusion

You've just broken down a live agent scenario and proposed how to make it smarter. Now, let's apply that same reflective lens to your own design instincts. In the next activity, Coursera Coach will help you evaluate whether your agent architecture is robust enough to handle complexity, edge cases, and user unpredictability.